1. Introduction

In this lab, students will build a simple MERN-stack application: a **"Creature Collector"** web app (Pokémon-inspired) where users can register/login and manage a list of creatures. The domain is minimal: users can add or remove creatures and view their names. The focus is on full-stack integration, not complex game logic.

Learning Objectives: After completing this lab, students will be able to:
- Set up a Node/Express backend with MongoDB (via Mongoose) for data storage.
- Implement JWT-based authentication, storing the token in an HTTP-only cookie, and manage sessions with *express-session*[1].
- Explain differences between JWT tokens (client-stored, stateless) and server-side sessions (stateful)[2][1], and why both are used for demonstration.
- Develop a React frontend (functional components with Hooks[3]) for login and a dashboard, calling the backend API.
- Use basic UI (no heavy libraries) to create a login/register form and a creature list with add/delete functionality.

Expected Final Output: A working web app where users can register an account, log in, and then see a dashboard listing their creatures. On the dashboard they can add a new creature (giving it a name) or delete an existing one. All backend calls use REST endpoints (e.g. **POST /auth/register**, **POST /auth/login**, **GET /creatures**, **POST /creatures**, **DELETE /creatures/:id**). The app uses JWT authentication (sent in an HTTP-only cookie) and also maintains a session on the server for illustrative purposes[2][4].

2. System Architecture

Diagram (Textual):

```
[React Frontend] <----> [Express/Node Backend] <----> [MongoDB (via
Mongoose)]
```

- The **Frontend (React)** displays the UI and communicates with the backend over HTTP (fetch/Axios).
- The **Backend (Express)** exposes REST endpoints and handles authentication. It uses **Mongoose** to interact with MongoDB for data persistence.
- **MongoDB** stores two collections: *users* and *creatures*. Each user can have multiple creatures (we store `userId` on each creature).

Key Flow: The user logs in via React → React sends credentials to `POST /auth/login` → Backend verifies user and issues a JWT (stored in an HTTP-only cookie) and creates a session on the server[1]. On success, the frontend loads the Dashboard. To fetch creatures, React calls `GET /creatures`, sending the cookie; the backend checks the session or JWT to identify the user and returns their creatures.

3. Prerequisites

- **Node.js:** Ensure Node.js (v18 or later) is installed. This lab was tested with Node 18+ (download from nodejs.org).
- **MongoDB:** You need a MongoDB database. You can install MongoDB locally or use a free cloud database (e.g. MongoDB Atlas). Obtain a connection string (URI) for Mongoose.
- **Tools:** A code editor (e.g. VSCode), terminal/command prompt, and either Postman or a browser for testing. Ensure you have `npm` available.

4. Step-by-Step Implementation

We break the work into phases. Total time ~6–8 hours; time estimates are in Section 7.

Phase 1: Backend Setup (Estimated ~1.5 hrs)

1. **Initialize Project:** Create a new folder `creature-collector` and run `npm init -y` to create `package.json`.
2. **Install Dependencies:** Inside the folder, install Express, Mongoose, and other needed packages:

   ```
   npm install express mongoose jsonwebtoken cookie-parser express-session bcrypt
   npm install --save-dev nodemon
   ```

3. **express:** web framework
4. **mongoose:** MongoDB ODM (object modeling)[5]
5. **jsonwebtoken:** for creating/verifying JWTs
6. **cookie-parser:** to parse cookies in requests
7. **express-session:** for session handling[1]
8. **bcrypt:** to hash passwords
9. **nodemon:** (dev) to auto-restart server on changes.
10. **Create Entry File:** Create `server.js` (or `index.js`). This will import Express and start the server. Add basic boilerplate:

    ```
    // server.js
    const express = require('express');
    const mongoose = require('mongoose');
    const session = require('express-session');
    const cookieParser = require('cookie-parser');
    const app = express();

    // Middleware
    app.use(express.json());
    app.use(cookieParser());
    app.use(session({
      secret: 'secret-key', // TODO: use env variable in real apps
      resave: false,
      saveUninitialized: true,
      cookie: { httpOnly: true }  // default: httpOnly true[4]
    }));

    // Connect to MongoDB
    mongoose.connect('mongodb://localhost:27017/creatureDB')
      .then(() => console.log('MongoDB connected'))
      .catch(err => console.error(err));

    // Basic route to check server
    app.get('/', (req, res) => {
      res.send('Hello from Creature Collector API');
    });

    // TODO: Add API routes (see Phase 4)
    ```

```
// Start server
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

11. We enabled `express.json()` to parse JSON bodies.
12. `express-session` is configured with a simple secret. Notice **session data is not saved in the cookie itself, just the session ID**[1]. The cookie is HTTP-only by default[4].
13. We will add API routes in Phase 4.
14. **Folder Structure:** Organize code into folders (for clarity, following MVC style[6]):

```
creature-collector/
├ controllers/
├ models/
├ routes/
├ middleware/
├ server.js
└ package.json
```

15. Put Mongoose schemas in `models/`, route handlers in `controllers/`, and route definitions in `routes/`.
16. We will reference this structure as we add code.

Phase 2: Database Design (Estimated ~1 hr)

Define Mongoose schemas (in `models/`). We use **2 schemas**: `User` and `Creature`.

1. **User Schema:** File `models/User.js`:

```
const mongoose = require('mongoose');
const userSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true },
  password: { type: String, required: true }
});
module.exports = mongoose.model('User', userSchema);
```

2. Here `username` is unique. The `password` will store the hashed password. Using Mongoose, we define each field's type[7].
3. **Creature Schema:** File `models/Creature.js`:

```
const mongoose = require('mongoose');
const creatureSchema = new mongoose.Schema({
  name: { type: String, required: true },
  power: String,
  owner: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }
```

```
});
module.exports = mongoose.model('Creature', creatureSchema);
```

4. Each creature has a `name` and (optional) `power`. The `owner` field stores the `User`'s `_id` to link a creature to its user.
5. We could query only a user's creatures via this relation.

Phase 3: Authentication (Estimated ~2 hrs)

Implement registration, login, JWT issuance, and session tracking.

1. **Password Hashing:** In `User` controller (e.g. `controllers/authController.js`), we will hash passwords using **bcrypt**. Example snippet:

```
const bcrypt = require('bcrypt');
// In register handler:
const hashed = await bcrypt.hash(req.body.password, 10);
// Save hashed password instead of plain text.
```

2. **JWT Creation:** Use `jsonwebtoken`. After login, generate a token:

```
const jwt = require('jsonwebtoken');
// After verifying user credentials:
const token = jwt.sign({ userId: user._id }, 'jwt-secret-key', {
expiresIn: '1h' });
// Send token to client in HTTP-only cookie
res.cookie('token', token, { httpOnly: true });
```

3. The `token` payload contains userId. The secret should be kept safe (env var in practice).
4. We send it in a cookie: `httpOnly: true` ensures JavaScript cannot read it, protecting it from XSS[4].
5. **Session Integration:** At login, also create a session variable to track logged-in status:

```
req.session.userId = user._id;
```

6. This stores `userId` in the server-side session store. Now each subsequent request (with the cookie holding the session ID) will have `req.session.userId` available.
7. **Differences (Explanation):**
8. *Sessions* (via express-session) keep server-side data. The cookie holds only a session ID; actual user info is on server[1].
9. *JWTs* are self-contained tokens sent to the client; the server does not need to store them[2][8]. They are stateless and scalable.
10. In this lab, we use both for learning: JWT in cookie ensures stateless auth (could verify token on each request), while session highlights stateful auth (storing session on server). We might check both for demonstration.

Phase 4: API Development (Estimated ~2 hrs)

Define routes and controllers for the 5 endpoints. Each endpoint description includes purpose, request/response, and code.

1. **POST `/auth/register` – Purpose:** Register a new user.
2. **Request:** `{ username, password }` (JSON).
3. **Response:** Success message or error.
4. **Code:** In `controllers/authController.js`:

```
const User = require('../models/User');
const bcrypt = require('bcrypt');
exports.register = async (req, res) => {
  try {
    const { username, password } = req.body;
    const hashed = await bcrypt.hash(password, 10);
    const newUser = new User({ username, password: hashed });
    await newUser.save();
    res.status(201).json({ message: 'User registered' });
  } catch (err) {
    res.status(400).json({ error: 'Username already exists' });
  }
};
```

5. **Route Setup:** In `routes/auth.js`:

```
const express = require('express');
const router = express.Router();
const authCtrl = require('../controllers/authController');
router.post('/register', authCtrl.register);
module.exports = router;
```

6. **POST `/auth/login` – Purpose:** Log in an existing user.
7. **Request:** `{ username, password }`.
8. **Response:** On success, set cookie and session, return user info or message.
9. **Code:** (add to `authController.js`)

```
exports.login = async (req, res) => {
  const { username, password } = req.body;
  const user = await User.findOne({ username });
  if (!user) return res.status(400).json({ error: 'Invalid credentials'
});
  const match = await bcrypt.compare(password, user.password);
  if (!match) return res.status(400).json({ error: 'Invalid
credentials' });
  // Create JWT
  const token = jwt.sign({ userId: user._id }, 'jwt-secret-key', {
expiresIn: '1h' });
  res.cookie('token', token, { httpOnly: true });
  // Create session
  req.session.userId = user._id;
```

```
  res.json({ message: 'Login successful' });
};
```

10. **Route:** In `routes/auth.js` add:

```
router.post('/login', authCtrl.login);
app.use('/auth', router);
```

11. **GET `/creatures` – Purpose:** Return list of creatures for the logged-in user.
12. **Request:** No body (but cookie/session must be present).
13. **Response:** `[ { _id, name, power }, ... ]`.
14. **Code:** In `controllers/creatureController.js`:

```
const Creature = require('../models/Creature');
exports.getCreatures = async (req, res) => {
  const userId = req.session.userId;
  if (!userId) return res.status(401).json({ error: 'Not logged in' });
  const creatures = await Creature.find({ owner: userId });
  res.json(creatures);
};
```

    - We retrieve `userId` from session (could also verify JWT from `req.cookies.token`).
15. **Route:** In `routes/creatures.js`:

```
const express = require('express');
const router = express.Router();
const creatureCtrl = require('../controllers/creatureController');
router.get('/', creatureCtrl.getCreatures);
module.exports = router;
```

16. **POST `/creatures` – Purpose:** Add a new creature.
17. **Request:** `{ name, power }` in JSON.
18. **Response:** The created creature object.
19. **Code:** (in `creatureController.js`)

```
exports.addCreature = async (req, res) => {
  const userId = req.session.userId;
  if (!userId) return res.status(401).json({ error: 'Not logged in' });
  const { name, power } = req.body;
  const newCreature = new Creature({ name, power, owner: userId });
  await newCreature.save();
  res.status(201).json(newCreature);
};
```

20. **Route:** In `routes/creatures.js` add:

```
router.post('/', creatureCtrl.addCreature);
```

21. **DELETE `/creatures/:id` – Purpose:** Remove a creature by ID (if owned by user).
22. **Request:** No body; ID in URL.

23. **Response:** Success message.
24. **Code:** (in `creatureController.js`)

```
exports.deleteCreature = async (req, res) => {
  const userId = req.session.userId;
  if (!userId) return res.status(401).json({ error: 'Not logged in' });
  const { id } = req.params;
  const creature = await Creature.findOneAndDelete({ _id: id, owner:
userId });
  if (!creature) return res.status(404).json({ error: 'Creature not
found' });
  res.json({ message: 'Creature deleted' });
};
```

25. **Route:** In `routes/creatures.js`:

```
router.delete('/:id', creatureCtrl.deleteCreature);
```

26. **Mount Routers:** In `server.js` after middleware:

```
const authRoutes = require('./routes/auth');
const creatureRoutes = require('./routes/creatures');
app.use('/auth', authRoutes);
app.use('/creatures', creatureRoutes);
```

Now the endpoints are active. Each one checks for a logged-in user via `req.session.userId`.

Phase 5: Frontend Setup (Estimated ~1 hr)

1. **Create React App:** In a new folder (sibling to server), run:

```
npx create-react-app creature-collector-client
cd creature-collector-client
npm install
```

This sets up a basic React app (CRA).

2. **Project Structure:** Under `src/`, create folders for components and services:

```
src/
├ api/
│   └ api.js
├ components/
│   ├ Login.js
│   ├ Register.js
│   └ Dashboard.js
├ App.js
└ index.js
```

3. `api/api.js` will hold functions to call our backend endpoints.
4. Components for Login, Register forms and Dashboard view.

Phase 6: UI Implementation (Estimated ~2 hrs)

a) API Helper (src/api/api.js)

Create a simple wrapper around `fetch` that includes cookies:

```
export async function apiRequest(path, options = {}) {
  const response = await fetch(`http://localhost:5000${path}`, {
    credentials: 'include', // send cookies
    headers: { 'Content-Type': 'application/json' },
    ...options
  });
  return response.json();
}
```

b) Login and Registration Components

1. **Register Component (components/Register.js):** A form to take username and password, send to `/auth/register`.

   ```
   import React, { useState } from 'react';
   import { apiRequest } from '../api/api';

   function Register() {
     const [username, setUsername] = useState('');
     const [password, setPassword] = useState('');

     const handleSubmit = async e => {
       e.preventDefault();
       const result = await apiRequest('/auth/register', {
         method: 'POST',
         body: JSON.stringify({ username, password })
       });
       alert(result.message || result.error);
     };

     return (
       <form onSubmit={handleSubmit}>
         <h2>Register</h2>
         <input value={username} onChange={e =>
   setUsername(e.target.value)} placeholder="Username" required />
         <input type="password" value={password} onChange={e =>
   setPassword(e.target.value)} placeholder="Password" required />
         <button type="submit">Register</button>
       </form>
     );
   }
   export default Register;
   ```

2. **Login Component (components/Login.js):** Similar form, calls `/auth/login`.

```
import React, { useState } from 'react';
import { apiRequest } from '../api/api';

function Login({ onLogin }) {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');

  const handleSubmit = async e => {
    e.preventDefault();
    const result = await apiRequest('/auth/login', {
      method: 'POST',
      body: JSON.stringify({ username, password })
    });
    if (result.message === 'Login successful') {
      onLogin(); // notify parent to show dashboard
    } else {
      alert(result.error);
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <h2>Login</h2>
      <input value={username} onChange={e =>
setUsername(e.target.value)} placeholder="Username" required />
      <input type="password" value={password} onChange={e =>
setPassword(e.target.value)} placeholder="Password" required />
      <button type="submit">Login</button>
    </form>
  );
}
export default Login;
```

3. **App Component (App.js):** Route between Login and Register (tabs or toggles) and show Dashboard after login.

```
import React, { useState } from 'react';
import Login from './components/Login';
import Register from './components/Register';
import Dashboard from './components/Dashboard';

function App() {
  const [userLoggedIn, setUserLoggedIn] = useState(false);
  const [showLogin, setShowLogin] = useState(true);

  if (userLoggedIn) {
    return <Dashboard />;
  }
```

```
      return (
        <div>
          {showLogin
            ? <Login onLogin={() => setUserLoggedIn(true)} />
            : <Register />}
          <button onClick={() => setShowLogin(!showLogin)}>
            {showLogin ? 'Go to Register' : 'Go to Login'}
          </button>
        </div>
      );
    }
    export default App;
```

c) Dashboard Component (components/Dashboard.js)

Display creature list and a form to add:

```
import React, { useState, useEffect } from 'react';
import { apiRequest } from '../api/api';

function Dashboard() {
  const [creatures, setCreatures] = useState([]);
  const [name, setName] = useState('');
  const [power, setPower] = useState('');

  // Fetch creatures on load
  useEffect(() => {
    apiRequest('/creatures').then(data => setCreatures(data));
  }, []);

  const addCreature = async e => {
    e.preventDefault();
    const newCreature = await apiRequest('/creatures', {
      method: 'POST',
      body: JSON.stringify({ name, power })
    });
    if (newCreature._id) {
      setCreatures([...creatures, newCreature]);
      setName(''); setPower('');
    } else {
      alert(newCreature.error);
    }
  };

  const deleteCreature = async (id) => {
    await apiRequest(`/creatures/${id}`, { method: 'DELETE' });
    setCreatures(creatures.filter(c => c._id !== id));
  };

  return (
```

```
    <div>
      <h2>Your Creatures</h2>
      <ul>
        {creatures.map(c => (
          <li key={c._id}>
            {c.name} (Power: {c.power})
            <button onClick={() => deleteCreature(c._id)}>Delete</button>
          </li>
        ))}
      </ul>
      <h3>Add New Creature</h3>
      <form onSubmit={addCreature}>
        <input value={name} onChange={e => setName(e.target.value)}
placeholder="Name" required />
        <input value={power} onChange={e => setPower(e.target.value)}
placeholder="Power" />
        <button type="submit">Add Creature</button>
      </form>
    </div>
  );
}
export default Dashboard;
```

- We fetch `/creatures` on mount and update state.
- Adding/deleting also calls the API and updates UI.

5. Testing Instructions

Use **Postman** or browser to test as you build.

- **Test Registration:** Send POST to `http://localhost:5000/auth/register` with JSON body `{"username":"ash","password":"pikachu"}`. Expect `201 Created` and a message.
- **Test Login:** POST to `/auth/login` with the same credentials. Verify in Postman that the response includes a `token` cookie (HTTP-only). The JSON response should say “Login successful”. Also check the cookie jar: a session cookie should appear.
- **Test Protected Endpoints:** Before login, `GET /creatures` should return error 401. After login (cookie present), `GET /creatures` should return an array (initially empty).
- **Add Creature:** POST to `/creatures` with `{"name":"Pikachu","power":"Electric"}`. Expect a 201 response with the new creature object (including `_id`).
- **View Creatures:** Now GET `/creatures` should list the added creature.

- **Delete Creature:** DELETE `/creatures/<id>` where `<id>` is the creature's `_id`. Expect a success message. Confirm by GET `/creatures` again (should be empty or missing that item).
- **Edge Cases:** Try adding a creature without being logged in (should be 401). Try registering a user that already exists (should 400 error).

6. Expected Output

- **Login Success:** After successful login, the frontend should navigate to the dashboard (as in the app flow). You should see a header "Your Creatures" with an empty list.
- **Creature List:** Each creature should appear as a list item. For example: `Pikachu (Power: Electric) [Delete]`
- **Add/Delete:** After adding a creature (via the form), it appears in the list. When you click its **Delete** button, it is removed from the list (and from the database).

The UI can be plain, for example:

```
Your Creatures
- Bulbasaur (Power: Grass)    [Delete]
- Charmander (Power: Fire)    [Delete]

Add New Creature
Name: [_______]  Power: [_______]  [Add Creature]
```

8. Stretch Goals

After completing the core lab, extend with additional features:

- **Creature Images:** Add an image URL field to the Creature schema and display an image in the list (using an `<img>` tag).
- **Search/Filter:** On the frontend, add a search bar to filter the creature list by name.

References: We followed Express and Mongoose guidelines (e.g. how to define schemas and middleware[7][1]) and React Hooks patterns[3]. Security aspects (HTTP-only cookies, session usage) align with best practices[4][2].

---

[1] [4] Express session middleware

https://expressjs.com/en/resources/middleware/session.html

[2] [8] Session-Based Authentication vs. JSON Web Tokens (JWTs) in System Design - GeeksforGeeks

https://www.geeksforgeeks.org/system-design/session-based-authentication-vs-json-web-tokens-jwts-in-system-design/

[3] Built-in React Hooks – React

https://react.dev/reference/react/hooks

[5] [7] Express Tutorial Part 3: Using a Database (with Mongoose) - Learn web development | MDN

https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs/mongoose

[6] Node.js project architecture best practices - LogRocket Blog

https://blog.logrocket.com/node-js-project-architecture-best-practices/